Centro Universitário de Excelência
Sistemas de Informação

Indexando os Dados: Árvore AVL, Busca O(log n) e Percurso Em-Ordem

Autores: Caio Josué Ribeiro Bastos, Camilly Vitória Ribeiro Ferreira, Davi Merces da Silva, Filipe Maia Archanjo dos Santos de Santana, Jackson Souza de Souza, Julia Gomes de Souza.

Feira de Santana, 2026

Agenda

O objetivo desta apresentação é demonstrar a substituição definitiva dos arrays por um Indexador AVL genérico, integrando os dados do CSV diretamente em árvores independentes por entidade para garantir buscas super rápidas.

1: O Nó Genérico e as Raízes Separadas

2. Inserção Direta e Busca O(log n)

3. Balanceamento e as Rotações AVL

4. Percurso Em-Ordem e Cast de Tipos

5. Considerações finais e Testes QA

O Nó Genérico e as Raízes Separadas

Substituição total dos arrays e listas por Árvores AVL na memória RAM.

O Nó armazena: ID (chave), registro genérico (`Object`), altura e referências esq/dir.

Para evitar colisão de IDs (ex: Cliente 1 e Peça 1), criamos raízes separadas no `Gerenciador` para cada entidade.

```
5  public class Gerenciador {
6
7      public static No raizPecas = null;
8      public static No raizServicos = null;
9      public static No raizClientes = null;
10     public static No raizOS = null;
11
12     private static int proximoCodigoPeca = 1;
13     private static int proximoCodigoServico = 1;
14
```

Inserção Direta e Busca O(log n)

O CSV agora carrega os dados inserindo-os diretamente na árvore correspondente, abandonando os arrays.

A busca navega comparando o ID desejado com o nó atual (menor vai para a esquerda, maior para a direita).

Complexidade O(log n): a cada passo, descartamos metade da árvore, garantindo alta performance.

```
129     private static Object buscarNo(No atual, int id) {
130         if (atual == null) {
131             return null;
132         }
133         if (id == atual.id) {
134             return atual.registro;
135         }
136         if (id < atual.id) {
137             return buscarNo(atual.esquerda, id);
138         }
139         return buscarNo(atual.direita, id);
140     }
141
```

Balanceamento e Rotações AVL

Fator de Balanceamento (FB) calculado por: `altura(esquerda) - altura(direita)`.

Valores permitidos rigorosamente restritos a -1, 0 e +1.

A altura é atualizada obrigatoriamente na "volta" da recursão de inserção.

Qualquer desbalanceamento aciona automaticamente uma das 4 rotações (simples ou duplas).

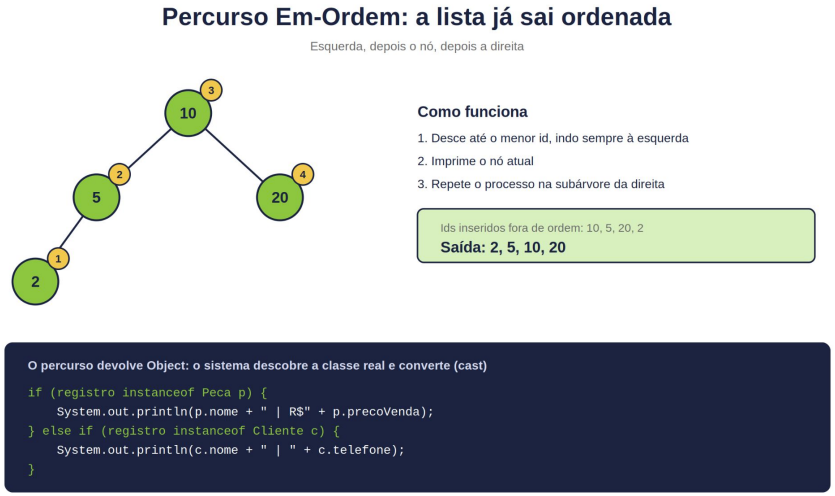
```
212     public static No rotacaoDireita(No y) {
213         No x = y.esquerda;
214         No meio = x.direita;
215
216         x.direita = y;
217         y.esquerda = meio;
218
219         y.altura = calcularAltura(y);
220         x.altura = calcularAltura(x);
221
222         return x;
223     }
224
```

Percurso Em-Ordem e Cast de Tipos

O Percurso Em-Ordem (Esquerda →Nó →Direita) gera relatórios automaticamente ordenados por ID crescente.

Elimina a necessidade de rodar algoritmos como *Bubble Sort* na hora de listar.

Como o nó guarda um Object, usamos instanceof para descobrir a classe real da entidade e fazer o *cast* (conversão).



Considerações finais

Evolução arquitetural: Saímos de uma busca linear lenta para um indexador profissional O(log n), sem "quebrar" as classes de negócio.

Integração concluída: O sistema agora lê do CSV direto para a árvore e busca direto da árvore. Arrays foram removidos.

Testes validados: Cenários de inserção, busca rápida, ordenação natural e rotação simples/dupla (QA-01 a QA-04) passaram com sucesso.

Referências

UNEX — Centro Universitário de Excelência. Guias das Sessões Tutoriais do OAT 3 (Encontros 1 a 4). Feira de Santana, 2026.

MecâniQA — Automotive Tech. Especificação do Produto Final — OAT 3: Indexando os meus dados. Feira de Santana, 2026.

Repositório do projeto:
github.com/Davibesxxol/mecaniQA-natal